

Assignment 1: Text Categorization

Nallathambi Vethiappan (s4577663) Luis Chial Sanchez (s4591321)
Faculty of Science, Leiden University, The Netherlands

1 Introduction

This report investigates multi-class text categorization using the 20newsgroups benchmark dataset in scikit-learn [3]. Building on the tutorial’s sparse feature extraction pipeline [1], we extend classification from four to all 20 categories for a robust evaluation. Our analysis compares classifiers and feature representations to assess their impact on performance, followed by targeted parameter tuning on the top combination. Evaluations employ macro-averaged Precision, Recall, and F1 scores to quantify efficacy across classes. The goal is to identify optimal configurations for effective text classification in sparse, high-dimensional settings, offering insights into feature weighting and model selection.

2 Methods

We adapted the scikit-learn tutorial for text classification [1], loading the full 20 newsgroups dataset with scikit-learn’s predefined train-test split as part of Task 1. All evaluations used macro-averaged Precision, Recall, and F1 from `classification_report` to assess multi-class performance.

Comparisons proceeded as follows:

- For Task 2 classifiers, we selected three from the tutorial’s eight options: Complement Naive Bayes (mandatory probabilistic baseline), Logistic Regression (linear discriminative model), and k-Nearest Neighbors (instance-based method), to contrast probabilistic, linear, and non-parametric approaches.
- For Task 3 features, we evaluated three representations with shared parameters (`max_df=0.5`, `min_df=5`, `stop_words='english'`): raw counts via `CountVectorizer`, term frequency (TF) via `TfidfVectorizer(use_idf=False)`, and TF-IDF via `TfidfVectorizer`. The top-performing pair advanced to parameter experiments.
- For Task 4 parameters experiments, on the best pair of classifier and feature from Task 3, we varied `TfidfVectorizer` settings:
 - (a) `Lowercasing` (`True/False`)
 - (b) `Stop_words` (`'english'/None`)
 - (c) `analyzer/ngram_range` (`word/(1,1)`, `word/(1,2)`, `word/(2,2)`, `char/(3,5)`)
 - (d) `max_features` (`None`, 5000, 10000, 20000).

The benchmark loop ensured consistent fitting, prediction, and metrics across all runs.

3 Results

3.1 Task 2 - Classifier Comparison

Table 1 reports macro-averaged Precision, Recall, and F1 for the three classifiers on the tutorial’s baseline TF-IDF vectorization.

Table 1: Macro-averaged Precision, Recall, and F1 for 3 classifiers

Classifier	Precision	Recall	F1
Complement Naive Bayes	0.829	0.820	0.819
Logistic Regression	0.847	0.841	<u>0.842</u>
k-Nearest Neighbors	0.772	0.755	0.753

Logistic Regression outperforms the others, as underlined in Table 1 with the highest F1 (0.842). This indicates that Logistic Regression, a linear model, excels in handling sparse text features, surpassing the probabilistic Naive Bayes baseline and instance-based kNN. kNN’s lower F1 suggests challenges with high-dimensional data.

3.2 Task 3 - Feature Comparison

Table 2 shows macro-averaged Precision, Recall, and F1 for the three classifiers across the three feature types Count, TF, and TF-IDF.

Table 2: Macro-averaged Precision, Recall, and F1 for 3 classifiers and 3 feature types

Classifier	Counts			TF			TF-IDF		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
Complement Naive Bayes	0.813	0.800	0.795	0.841	0.829	0.828	0.829	0.820	0.819
Logistic Regression	0.798	0.792	0.793	0.819	0.814	0.814	0.847	0.841	<u>0.842</u>
k-Nearest Neighbors	0.613	0.316	0.313	0.718	0.697	0.694	0.772	0.755	0.753

TF-IDF yields the strongest results, achieving the highest F1 (0.842) as underlined in Table 2 for Logistic Regression, improving 3-5% over counts through inverse document frequency weighting, which downplays common terms and boosts rare terms, benefiting all classifiers but most notably Logistic Regression. Raw counts underperform due to unnormalized frequencies favoring long documents.

3.3 Task 4 - Parameter Experiments

Table 3 presents macro-averaged Precision, Recall, and F1 from varying TfidfVectorizer parameters on the best combination from Task 3 (Logistic Regression + TF-IDF).

Table 3: Parameter experiments on Logistic Regression + TF-IDF

Parameter	Setting	Precision	Recall	F1
a. Lowercasing	True	0.847	0.841	0.842
	False	0.846	0.839	0.840
b. Stop words	With 'english'	0.847	0.841	0.842
	Without	0.848	0.841	0.842
c. Analyzer + ngram_range	word, (1,1)	0.847	0.841	0.842
	word, (1,2)	0.851	0.845	0.846
	word, (2,2)	0.724	0.709	0.713
	char, (3,5)	0.855	0.849	<u>0.850</u>
d. max_features	All	0.847	0.841	0.842
	5000	0.807	0.802	0.803
	10000	0.831	0.824	0.825
	20000	0.845	0.838	0.839

Table 3 indicates that char ngrams (3-5) achieve the peak F1 (underlined), capturing subword patterns effectively, while word bigrams offer a modest gain (+0.004). Stop words show minimal impact, and reduced max_features degrades performance by limiting vocabulary.

4 Discussion

Logistic Regression with TF-IDF emerged as the top performer overall, attaining an F1 score of 0.842 and surpassing Naive Bayes by 2.3% and kNN by 8.9% in Tasks 2-3. This outcome underscores Logistic Regression’s proficiency in modeling sparse, high-dimensional text features, complemented by TF-IDF’s inverse document frequency mechanism, which diminishes the influence of common terms and elevates discriminative ones, thereby enhancing topical distinction across the 20 categories. In contrast, kNN exhibited diminished efficacy, attributed to the challenges of high dimensionality, while Naive Bayes served as a reliable probabilistic foundation yet fell short of Logistic Regression’s discriminative capabilities.

Task 4 experiments on this combination revealed char ngrams (3-5) as the strongest variant (F1=0.850), likely owing to their ability to capture subword structures and spelling mistakes that occur in informal newsgroup text. Word bigrams contributed a modest improvement (+0.004 F1) by incorporating multi-word expressions, whereas bigrams alone (2,2) substantially underperformed (0.713) due to the absence of unigram context. Stop words exerted negligible effects, and limiting max_features to 20,000 resulted in a 1.7% F1 decline by constraining the vocabulary. Collectively, these findings affirm Logistic Regression + TF-IDF with char ngrams as an optimal configuration for sparse text classification.

5 Reflection

Nallathambi handled Tasks 1 and 2 (data loading and classifier comparison), along with drafting the report. Luis managed Tasks 3 and 4 (feature comparison and parameter experiments), and reviewed/finalized the report for clarity and formatting.

We used ChatGPT assistant for (a) grasping differences among the 8 classifiers used in the tutorial, (b) calculating percentage improvements/declines (e.g., TF-IDF’s 3-5% F1 gain over counts) for reporting, and (c) formatting a multi-level table structure in the Latex code. It was helpful for rapid clarification and worked reliably overall. Minor issues such as overlapping table structures were corrected manually by referring to Tex formatting guidelines [2].

6 Conclusion

This study explored multi-class text categorization on the 20 Newsgroups dataset by extending the baseline example to all twenty categories. The experiments compared different classifiers and feature types to understand their impact on performance. Results showed that discriminative linear models, particularly Logistic Regression, performed strongly when paired with effective feature weighting. TF-IDF helped balance term importance and improved generalization across diverse topics, while character n-grams made the model more robust to spelling and word-form variations. Overall, the findings suggest that well-tuned linear models with thoughtful feature engineering can match the performance of more complex methods, providing a solid and interpretable baseline for future text classification research.

References

- [1] Classification of text documents using sparse features. https://scikit-learn.org/stable/auto_examples/text/plot_document_classification_20newsgroups.html. Accessed: 2025-09-30.
- [2] Tex latex table with multirow structure. <https://tex.stackexchange.com/questions/715091/latex-table-with-complex-multirow-structure-including-intermittent-lines>. Accessed: 2025-10-04.
- [3] Text feature extraction. https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction. Accessed: 2025-09-30.